

Documentation du pipeline Spark

Construction du graphe de co-vues — fonctions d'extraction

Neil Braun

Mirakl — Data Science / Target2Sell

Août 2026

1 Extraction des référentiels

1.1 get_customer

Résout, pour un client donné, son `publicId` T2S et son `publisherId` Artemis.

Signature.

```
def get_customer(spark: SparkSession, customer_shortcode: str) -> DataFrame:
```

Arguments.

- `spark` (`SparkSession`) — session utilisée pour ouvrir les tables du catalogue.
- `customer_shortcode` (`str`) — nom court du client, par exemple `maisonsdumonde`. Sert de filtre unique sur le référentiel.

Tables lues.

- `mirakl_ai.ds_etl_prod.t2s_gold_customer` — référentiel des clients T2S. Colonnes utilisées : `shortName` (filtre), `publicId`, `databaseName`.
- `mirakl_data_platform.prod_data_platform_silver.artemis_insights_customer` — clients côté Artemis. Colonnes utilisées : `id`, `public_id`, `__k8s_namespace` (filtre d'environnement).
- `mirakl_data_platform.prod_data_platform_silver.artemis_publisher` — *publishers* Artemis. Colonnes utilisées : `id`, `insights_customer_id` (clé de jointure).

Traitement.

- `customers` : client T2S filtré sur `shortName`, projeté sur `publicId` et `databaseName` (renommé `db_name`).
- `publishers` : clients Artemis rattachés aux *publishers* par jointure `left` sur `insights_customer_id`, restreints au namespace `artemis-prod`, projetés sur `publisherId` et `public_id`.
- Jointure finale `inner customers/publishers` sur `publicId = public_id`; sélection des quatre colonnes de sortie.

Table produite. Une ligne par couple client / *publisher* ; unicité non garantie par le code. Types déduits du nommage, non relus dans le catalogue.

Colonne	Type	Provenance et sens
<code>customer_shortcode</code>	<code>string</code>	<code>t2s_gold_customer.shortName</code> ; copie de l'argument
<code>customerId</code>	<code>string</code>	<code>t2s_gold_customer.publicId</code> ; identifiant public T2S
<code>db_name</code>	<code>string</code>	<code>t2s_gold_customer.databaseName</code> ; base à interroger en aval
<code>publisherId</code>	<code>long</code>	<code>artemis_publisher.id</code> ; nul si le client n'a pas de <i>publisher</i>

Complètent le référentiel client par le catalogue produit et les embeddings, à partir des mêmes `db_names`.

1.2 get_products

Charge les produits maîtres actifs et diffusables (`parentId == -1`) du catalogue, avec leurs métadonnées d’affichage.

Signature.

```
def get_products(spark: SparkSession, db_names: list[str]) -> DataFrame:
```

Arguments.

- `spark` (`SparkSession`).
- `db_names` (`list[str]`) — bases filtrées via `isin`.

Tables lues.

- `mirakl_ai.ds_etl_prod.t2s_mongo_product_0_current` — catalogue produit. Colonnes utilisées : `db_name`, `isActive`, `isOkInStream`, `parentId` (filtres), `internalId`, `fwProductId`, `name`, `imageUrl`.

Traitement.

- Filtre le catalogue sur `db_name.isin(db_names)`, `isActive`, `isOkInStream` et `parentId == -1` (produit maître); cast `internalId` en `bigint` et projection des quatre colonnes de sortie.

Table produite. Une ligne par produit maître actif. Types non castés déduits du nommage, non relus dans le catalogue.

Colonne	Type	Provenance et sens
<code>internalId</code>	<code>bigint</code>	cast explicite depuis <code>t2s_mongo_product_0_current.internalId</code>
<code>fwProductId</code>	<code>string</code>	<code>t2s_mongo_product_0_current.fwProductId</code>
<code>name</code>	<code>string</code>	<code>t2s_mongo_product_0_current.name</code>
<code>imageUrl</code>	<code>string</code>	<code>t2s_mongo_product_0_current.imageUrl</code>

1.3 get_embeddings

Charge les embeddings d’un modèle donné (`last_model_id` fixé dans le code) pour les produits des clients demandés.

Signature.

```
def get_embeddings(spark: SparkSession, db_names: list[str]) -> DataFrame:
```

Arguments.

- `spark` (`SparkSession`).
- `db_names` (`list[str]`) — bases filtrées via `isin`.

Tables lues.

- `mirakl_ai.ds_artemis_prod.product_embeddings` — embeddings produit. Colonnes utilisées : `db_name`, `last_model_id` (filtres), `internalId`, `embeddings`, `last_update_date`.

Traitement.

— Filtre les embeddings sur `db_name.isin(db_names)` et `last_model_id == "06bbe5791147457aa395960da021c05c"` (modèle codé en dur) ; projection de `internalId`, `embeddings`, `last_update_date`.

Table produite. Une ligne par `internalId`, sous réserve d'un seul embedding par produit pour ce `last_model_id` — non garanti par le code. Le docstring évoque des produits actifs, mais aucun filtre `isActive/parentId` n'apparaît ici : ce filtrage, s'il existe, vient d'amont (table déjà restreinte, ou jointure ultérieure avec 1.2).

Colonne	Type	Provenance et sens
<code>internalId</code>	<code>bigint</code>	<code>product_embeddings.internalId</code>
<code>embeddings</code>	<code>array<double></code>	<code>product_embeddings.embeddings</code> ; vecteur de features
<code>last_update_date</code>	<code>timestamp</code>	<code>product_embeddings.last_update_date</code>

2 Tracking, sessionisation et graphe de co-vues

Le pipeline enchaîne `get_views` → `sessionize_views` → `split_sessions` → `build_co_view_edges` → `remove_train_edges`.

2.1 get_views

Charge le tracking brut (non nettoyé) des vues produit, avec réconciliation de l'identifiant utilisateur via la table des utilisateurs fusionnés.

Signature.

```
def get_views(spark: SparkSession, customer_ids: list[str], start_date: date, end_date: date) -> DataFrame:
```

Arguments.

— `spark` (`SparkSession`).
— `customer_ids` (`list[str]`) — clients filtrés via `isin`.
— `start_date`, `end_date` (`date`) — bornes de `emitted.between(...)`, incluses.

Tables lues.

— `mirakl_data_platform.prod_data_platform_silver.t2s_merged_user` — réconciliation d'identifiants utilisateur. Colonnes utilisées : `customerId` (filtre), `masterId`, `sourceId`.
— `mirakl_data_platform.prod_data_platform_silver.t2s_tracking_product_display_page_event_fct` — événements de vue produit. Colonnes utilisées : `customerId`, `emitted`, `user.internalId`, `masterProductInternalId` (filtres et projections).

Traitement.

— `df_merged_user` : clients filtrés sur `customerId`, projetés sur `masterId` → `userMasterId` et `sourceId` → `userId`.
— Bloc principal : événements de tracking filtrés sur `customerId`, `emitted` (fenêtre `start_date-end_date`), `user.internalId` et `masterProductInternalId` non nuls ; `masterProductInternalId` renommé `internalId` ; `userId` rattaché à l'id maître par jointure `left + coalesce(userMasterId, userId)` avec `df_merged_user`.

Table produite. Une ligne par événement de vue, non dédoublée.

Colonne	Type	Provenance et sens
emitted	timestamp	horodatage de l'événement
internalId	bigint	masterProductInternalId renommé
userId	bigint	id utilisateur réconcilié (user.internalId ou masterId), sert uniquement à l'agrégation en aval

2.2 sessionize_views

Découpe les vues d'un utilisateur en sessions à partir d'un délai d'inactivité, et ajoute `session_id` et `session_start` à chaque vue.

Signature.

```
def sessionize_views(df_views: DataFrame, session_timeout: int = 1800) -> DataFrame:
```

Arguments.

- `df_views` (DataFrame) — produit par `get_views` (2.1). Colonnes attendues : `userId`, `internalId`, `emitted`.
- `session_timeout` (int, défaut 1800) — délai d'inactivité en secondes au-delà duquel une nouvelle session démarre.

Traitement.

- `seconds_since_last` : écart en secondes avec la vue précédente du même utilisateur (fenêtre `partitionBy(userId).orderBy(emitted, internalId)`).
- `is_new_session` : vaut 1 si première vue ou écart > `session_timeout`, sinon 0.
- `session_id` : somme cumulée de `is_new_session` sur la fenêtre utilisateur — numérote les sessions.
- `session_start` : `min(emitted)` de la session.

Table produite. Une ligne par vue (même granularité que 2.1), enrichie de `session_id` et `session_start`.

Colonne	Type	Provenance et sens
userId	bigint	recopié de <code>df_views</code>
session_id	long	compteur cumulé de nouvelles sessions par utilisateur
internalId	bigint	recopié de <code>df_views</code>
emitted	timestamp	recopié de <code>df_views</code>
session_start	timestamp	<code>min(emitted)</code> de la session

2.3 split_sessions

Assigne chaque session à `train`, `val` ou `test` selon deux dates seuils, et reporte ce `split` sur chaque vue.

Signature.

```
def split_sessions(df_sessionized: DataFrame, cutoff_train: date, cutoff_val: date) -> DataFrame:
```

Arguments.

- `df_sessionized` (DataFrame) — produit par `sessionize_views` (2.2).
- `cutoff_train`, `cutoff_val` (date) — seuils de `session_start` : $< \text{cutoff_train} \rightarrow \text{train}$, entre les deux $\rightarrow \text{val}$, $\geq \text{cutoff_val} \rightarrow \text{test}$.

Traitement.

- `df_sessions` : sessions dédoublées (`userId`, `session_id`, `session_start`), étiquetées `split` selon les seuils `cutoff_train` / `cutoff_val`.
- Jointure `inner` pour reporter `split` sur chaque vue de 2.2.

Table produite. Même granularité que 2.2 (une ligne par vue), avec la colonne `split` en plus ; les autres colonnes sont inchangées.

Colonne	Type	Provenance et sens
<code>split</code>	<code>string</code>	<code>train</code> / <code>val</code> / <code>test</code> , dérivé de <code>session_start</code>

2.4 build_co_view_edges

Construit les arêtes de co-vue : pour chaque paire de produits vus dans la même session, un poids et un nombre d'occurrences.

Signature.

```
def build_co_view_edges(df_sessionized: DataFrame) -> DataFrame:
```

Arguments.

- `df_sessionized` (DataFrame) — typiquement un sous-ensemble de 2.3 filtré sur un `split`. Colonnes attendues : `userId`, `session_id`, `internalId`.
- Contrôlé par la constante module `USE_SESSION_WEIGHT_NORM` (bool, actuellement `False`) : bascule le calcul de `pair_weight` entre poids uniforme et normalisation par taille de session.

Traitement.

- `df_dedup` : vues dédoublées sur (`userId`, `session_id`, `internalId`), enrichies de `session_size` (nombre de produits distincts de la session).
- `df_pairs` : auto-jointure `inner` sur (`userId`, `session_id`) avec `L.internalId < R.internalId` pour générer les paires non ordonnées, chacune pondérée par `pair_weight` (uniforme, ou normalisé par `session_size` si `USE_SESSION_WEIGHT_NORM`).
- Agrégation finale par (`product_A`, `product_B`) : somme des poids (`weight`) et comptage des occurrences (`raw_count`).

Table produite. Une ligne par paire de produits co-vus, non orientée (`product_A < product_B`).

Colonne	Type	Provenance et sens
<code>product_A</code>	<code>bigint</code>	<code>internalId</code> le plus petit de la paire
<code>product_B</code>	<code>bigint</code>	<code>internalId</code> le plus grand de la paire
<code>weight</code>	<code>double</code>	somme de <code>pair_weight</code> sur les sessions co-occurentes
<code>raw_count</code>	<code>long</code>	nombre de sessions où la paire co-occure

2.5 remove_train_edges

Retire de `df_edges` toute paire déjà présente dans `df_train_edges`.

Signature.

```
def remove_train_edges(df_edges: DataFrame, df_train_edges: DataFrame) -> DataFrame:
```

Arguments.

- `df_edges` (DataFrame) — arêtes produites par 2.4 (typiquement `val` ou `test`).
- `df_train_edges` (DataFrame) — arêtes d'entraînement, même origine. Colonnes utilisées : `product_A`, `product_B`.

Traitement.

- Jointure `left_anti` sur (`product_A`, `product_B`).

Table produite. Même schéma que 2.4, restreint aux paires absentes de `df_train_edges`.

2.6 build

Enchaîne `build_co_view_edges` et, si des arêtes d'entraînement sont fournies, `remove_train_edges`, pour produire les arêtes d'un split.

Signature.

```
def build(df_subset, df_train_edges=None):
```

Arguments.

- `df_subset` (non annoté, DataFrame) — sous-ensemble de 2.3 filtré sur un `split`. Colonnes attendues : `userId`, `session_id`, `internalId`.
- `df_train_edges` (non annoté, DataFrame, défaut `None`) — arêtes d'entraînement déjà construites, à exclure du résultat.

Traitement.

- Construit les arêtes de co-vue de `df_subset` via `build_co_view_edges` (2.4).
- Si `df_train_edges` n'est pas `None`, retire de ces arêtes celles déjà présentes dans `df_train_edges` via `remove_train_edges` (2.5).

Table produite. Même schéma que 2.4. Dans l'appel observé, `build` sert aux trois splits : sans `df_train_edges` pour `train`, puis avec les arêtes de `train` pour `val` et pour `test`. Seule la fuite avec `train` est donc retirée ; une arête commune à `val` et `test` n'est éliminée par aucun des deux appels.

2.7 count_nodes

Compte le nombre de nœuds distincts apparaissant dans au moins une arête de `df_edges`.

Signature.

```
def count_nodes(df_edges: DataFrame) -> int:
```

Arguments.

- `df_edges` (DataFrame) — arêtes, typiquement produites par `build` (2.6) ou par 2.4. Colonnes utilisées : `product_A`, `product_B`.

Traitement.

— Union des colonnes `product_A` et `product_B` (renommées `node`), dédoublée (`distinct`) — ensemble des nœuds apparaissant dans au moins une arête.

Sortie. Retourne un entier (`count()` sur le résultat), pas un `DataFrame` : le nombre de nœuds distincts du graphe défini par `df_edges`.

3 Remappage des identifiants et mise en forme tensorielle (pandas / PyTorch)

Le pipeline bascule ici en `pandas` (conversion depuis `Spark` non visible dans ce fragment) : `build_node_mapping` → `remap_edges` → `edges_to_tensors`.

3.1 build_node_mapping

Construit, à partir d'un nombre arbitraire de `DataFrames` d'arêtes, une table de correspondance `internalId` → index contigu (`0..N-1`).

Signature.

```
def build_node_mapping(*edge_dfs: pd.DataFrame) -> dict[int, int]:
```

Arguments.

— `*edge_dfs` (`pd.DataFrame`, variadique) — un ou plusieurs `DataFrames` d'arêtes (`train`, `val`, `test`), chacun attendu avec les colonnes `product_A`, `product_B`.

Traitement.

— `all_nodes` : identifiants uniques de `product_A` / `product_B` de tous les `edge_dfs`, triés par valeur.
— `node2idx` : index contigu (`enumerate`) sur ces identifiants triés — détermine quel que soit l'ordre des `edge_dfs` passés.

Sortie. Retourne un `dict[int, int]`, pas un `DataFrame` : `internalId` → index contigu `0..N-1`. Le tri préalable rend le mapping déterministe quel que soit l'ordre des `edge_dfs` passés, tant que leur ensemble est le même.

3.2 remap_edges

Remplace `product_A` et `product_B` par leurs index contigus dans un `DataFrame` d'arêtes.

Signature.

```
def remap_edges(df: pd.DataFrame, node2idx: dict[int, int]) -> pd.DataFrame:
```

Arguments.

— `df` (`pd.DataFrame`) — `DataFrame` d'arêtes, colonnes `product_A`, `product_B` attendues.
— `node2idx` (`dict[int, int]`) — mapping produit par `build_node_mapping` (3.1).

Traitement.

— Copie `df`, puis remplace `product_A` et `product_B` par leur image dans `node2idx` (`.map`).

Table produite. Même schéma que `df` en entrée ; `product_A` et `product_B` portent désormais des index contigus, les autres colonnes sont inchangées. Un `internalId` absent de `node2idx` n'est pas signalé : `.map` produit silencieusement `NaN` pour toute clé manquante.

3.3 edges_to_tensors

Convertit un `DataFrame` d'arêtes remappées en tensors `edge_index` / `edge_attr` au format PyG.

Signature.

```
def edges_to_tensors(df: pd.DataFrame, symmetric: bool = True) -> tuple[torch.Tensor, torch.Tensor]:
```

Arguments.

- `df` (`pd.DataFrame`) — arêtes remappées, produites par `remap_edges` (3.2). Colonnes utilisées : `product_A`, `product_B`, `weight`.
- `symmetric` (`bool`, défaut `True`) — si vrai, ajoute chaque arête dans les deux sens ($A \rightarrow B$ et $B \rightarrow A$).

Traitement.

- Extrait `src`, `dst`, `weight` des colonnes `product_A`, `product_B`, `weight`.
- Si `symmetric` : empile les deux sens ($src \rightarrow dst$ et $dst \rightarrow src$) et duplique les poids ; sinon un seul sens.
- Cast `edge_index` en `torch.long`, `edge_attr` en `torch.float`.

Sortie. Retourne le tuple (`edge_index`, `edge_attr`), pas un `DataFrame`. `edge_index` est de forme `[2, E]` si `symmetric=False`, ou `[2, 2E]` si `symmetric=True` (E = nombre de lignes de `df`) ; `edge_attr` est de forme assortie (`[E]` ou `[2E]`). Le docstring annonce `[2, E]` et `[E]` dans tous les cas, ce qui n'est exact que pour `symmetric=False`.

4 Rapprochement vues et exécutions de recommandation (ads)

4.1 recommended_products_for_t2s_user

Associe chaque vue produit trackée à l'exécution de recommandation (`adlog`) qui l'a vraisemblablement générée, en appariant sur produit, cookie, client et une fenêtre de 3 secondes autour de l'horodatage.

Signature.

```
def recommended_products_for_t2s_user(  
    spark: SparkSession,  
    customer_ids: list[str],  
    publisher_ids: list[str],  
    start_date: date,  
    end_date: date,  
) -> DataFrame:
```

Arguments.

- `spark` (`SparkSession`).
- `customer_ids` (`list[str]`) — clients filtrés via `isin`, côté tracking.
- `publisher_ids` (`list[str]`) — *publishers* filtrés via `isin`, côté `adlog`.
- `start_date`, `end_date` (`date`) — bornes de `__log_date.between(...)` et de `emitted.between(...)`, incluses.

Tables lues.

- `mirakl_data_platform.prod_data_platform_silver.t2s_merged_user` — même table que 2.1, même construction (`customerId` en filtre, `masterId/sourceId` projetés).
- `mirakl_data_platform.prod_data_platform_silver.ads_adlog_fct` (adlog) — logs du serveur de recommandation. Colonnes utilisées : `__k8s_namespace`, `publisherId`, `__log_date`, `pageType` (filtres), `productId`, `userCookie`, `customerId`, `logInstant` (clés/fenêtre de jointure), `executionId`, `searchTerm`, `userOrganizeRank`, `relevantProducts`, `sponsoredProductPlacementExecutions`, `productsReturned` (sortie).
- `mirakl_data_platform.prod_data_platform_silver.t2s_tracking_product_display_page_event_fct` — même table que 2.1, avec ici deux colonnes supplémentaires en clé de jointure : `fwProductId`, `user.cookie`.

Traitement.

- `df_merged_user` : même construction que 2.1.
- `df_t2s_recommended_products` : logs adlog filtrés (namespace `target2sell-prod`, `publisherId`, `__log_date`, `pageType` == "PRODUCT"), joints en inner à la table de tracking (elle-même filtrée sur `customerId`, `emitted`, et deux colonnes non nulles) sur produit / cookie / client et une fenêtre de $\pm 3s$ autour de `logInstant` ; `userId` rattaché à l'id maître comme dans 2.1 ; sélection des colonnes de sortie.

Table produite. Une ligne par paire (adlog, t2s) appariée par la jointure inner. Le prédicat ne fixe pas `emitted == logInstant` mais une fenêtre de tolérance de 3 secondes : si plusieurs exécutions adlog tombent dans cette fenêtre pour le même produit / cookie / client, le même événement de vue apparaît en plusieurs lignes — l'unicité par vue n'est pas garantie par le code.

Colonne	Type	Provenance et sens
<code>internalId</code>	<code>bigint</code>	<code>t2s.masterProductInternalId</code> renommé
<code>emitted</code>	<code>timestamp</code>	<code>t2s.emitted</code> ; horodatage de la vue
<code>userId</code>	<code>bigint</code>	id utilisateur réconcilié (même logique que 2.1)
<code>customerId</code>	<code>string</code>	<code>t2s.customerId</code> renommé
<code>executionId</code>	<code>string</code>	<code>adlog.executionId</code>
<code>logInstant</code>	<code>timestamp</code>	<code>adlog.logInstant</code> ; horodatage de l'exécution
<code>pageType</code>	<code>string</code>	<code>adlog.pageType</code> ; toujours "PRODUCT" après le filtre
<code>searchTerm</code>	<code>string</code>	<code>adlog.searchTerm</code>
<code>userOrganizeRank</code>	<code>string</code>	<code>adlog.userOrganizeRank</code> — une chaîne malgré le nom, pas un entier
<code>relevantProducts</code>	<code>array<struct></code>	<code>adlog.relevantProducts</code> au niveau racine, struct <code><fwProductId, relevanceScore, organizeScore, bidPrice></code> — différent du <code>relevantProducts</code> imbriqué dans <code>sponsoredProductPlacementExecutions</code> utilisé par 5.1
<code>sponsoredProductPlacementExecutions</code>	<code>array<struct></code>	<code>adlog.sponsoredProductPlacementExecutions</code> ; structure imbriquée, voir 5.1 pour les champs exploités
<code>productsReturned</code>	<code>array<struct></code>	<code>adlog.productsReturned</code> , struct <code><fwProductId, adId, adUnitId></code>

5 Construction des arêtes positives et négatives d'entraînement

5.1 positive_negative_edge_construction

Construit, pour un split donné, les arêtes positives et négatives utilisées pour l'entraînement et l'évaluation du modèle de lien trigger → candidat, à partir des exécutions de recommandation (4.1) et de la vérité de session (2.3). Une paire est positive si le produit candidat a effectivement été vu dans la session du trigger ; elle est négative si le candidat n'a pas été vu *et* n'est pas déjà une arête du graphe d'entraînement (df_train_edges), pour éviter les faux négatifs. Au passage, la fonction calcule aussi le MRR de l'algorithme de production (deux variantes) sur les mêmes exécutions, à titre de métrique de référence.

Signature.

```
def positive_negative_edge_construction(df_recommended: DataFrame, df_split: DataFrame, df_train_edges: DataFrame, node2idx: dict[int, int], split: str) -> tuple:
```

Arguments.

Argument	Type	Rôle
df_recommended	DataFrame	Exécutions de recommandation enrichies. Doit porter <code>userId</code> , <code>session_id</code> , <code>internalId</code> , <code>executionId</code> , <code>sponsoredProductPlacementExecutions</code> , <code>split</code> — <code>session_id</code> et <code>split</code> ne figurent pas dans le schéma documenté de 4.1 : ce DataFrame est donc le résultat d'une jointure amont (avec la sortie de 2.3, vraisemblablement sur <code>userId</code> / <code>internalId</code> / <code>emitted</code>) non visible dans ce fragment.
df_split	DataFrame	Sortie de 2.3 (ou son équivalent mis en cache) : une ligne par vue, avec <code>userId</code> , <code>session_id</code> , <code>internalId</code> , <code>split</code> . Sert de vérité de session — les produits réellement vus.
df_train_edges	DataFrame	Arêtes de co-vue d'entraînement, issues de 2.6 / 2.4. Colonnes utilisées : <code>product_A</code> , <code>product_B</code> . Sert à exclure les faux négatifs.
node2idx	dict[int, int]	Mapping <code>internalId</code> → index contigu, produit par <code>build_node_mapping</code> (3.1).
split	str	Split à traiter (" <code>train</code> " / " <code>val</code> " / " <code>test</code> "); filtre <code>df_recommended</code> et <code>df_split</code> .

Traitement.

- `df_recommended_products_t2s_with_sessions` : jointure inner sur (`userId`, `session_id`) entre les triggers de `df_recommended` et les vues de session de `df_split` — associe chaque trigger à tous les produits vus dans sa session.
- `df_recommended_exploded` : expose les placements (`sponsoredProductPlacementExecutions`) puis leurs douze premiers candidats (`relevantProducts`), avec le rang d'affichage 1-based (`rank_candidate`) — mis en `cache()`, réutilisé cinq fois en aval.
- `positive_candidates` : candidats dont `internalId_candidate` correspond à un produit vu en session — les *hits*, dédupliqués en (`product_A`, `product_B`) par exécution.
- `matched_rank` / MRR de production : meilleur rang par exécution converti en `reciprocal_rank`; `production_mrr_matched` moyenne sur les exécutions avec hit (comparable à `MRR_trigger`), `production_mrr_coverage` moyenne sur toutes les exécutions.
- `df_all_candidates` : tous les couples (`executionId`, `product_A`, `product_B`), positifs et négatifs confondus, avec le rang minimal conservé.

- `df_existing_pairs` : arêtes de `df_train_edges` dans les deux sens — pour tester l'appartenance au graphe sans ordre.
- `df_negative_labeled` : marque chaque candidat comme `is_existing_edge` ou non, par jointure `left` avec `df_existing_pairs`.
- `negative_candidates` : candidats absents du graphe d'entraînement, purgés des positifs déjà trouvés (`left_anti`) — mis en `cache()`.
- `deepest_pos_rank` : rang du positif le plus profond par exécution.
- `negative_candidates_matched` : négatifs limités au rang du positif le plus profond de leur exécution.
- `exec_only_negatives` : exécutions sans aucun positif, échantillonnées à `fraction=0.0` — donc actuellement vide.
- `negative_candidates_pure_negative` / `negative_candidates_sampled` : négatifs des exécutions sans positif (vide) unis aux négatifs des exécutions avec positif.
- Conversion pandas séparée des positifs et des négatifs (pas de *cross-join*).
- `exec2code` / `code2exec` : code contigu par `executionId`, sur toutes les exécutions du split, trié pour être déterministe.
- Remappage de `product_A` / `product_B` via `node2idx` et de `executionId` via `exec2code`, puis `dropna` des lignes non mappées.
- `pos_edge_index` / `neg_edge_index` : tenseurs `[3, N]` (`exec_code`, `product_A`, `product_B`), en `torch.long`.

Points d'attention.

- `df_recommended` doit déjà contenir `session_id` et `split`, absents du schéma documenté de 4.1 : une jointure amont non visible ici enrichit ce DataFrame avant l'appel.
- Avec `exec_only_negatives` échantillonné à `fraction=0.0`, les exécutions sans aucun positif ne contribuent *aucun* négatif d'entraînement dans la configuration actuelle du code, bien qu'elles reçoivent tout de même un `exec_code`.
- Le `dropna` final supprime silencieusement toute ligne dont `product_A`, `product_B` ou `exec_code` n'a pas pu être mappé, sans journalisation du nombre de lignes perdues.

Sortie. Retourne un tuple de six éléments (le type de retour annoté, `tuple`, n'est pas paramétré) :

Élément	Type	Contenu
<code>pos_edge_index</code>	<code>torch.Tensor</code>	forme <code>[3, N]</code> ; lignes (<code>exec_code</code> , <code>product_A</code> , <code>product_B</code>) des candidats positifs.
<code>neg_edge_index</code>	<code>torch.Tensor</code>	forme <code>[3, N]</code> ; même structure, candidats négatifs échantillonnés.
<code>production_mrr_matched</code>	<code>float</code>	MRR de production moyenné sur les seules exécutions ayant un hit ; comparable à <code>MRR_trigger</code> du modèle.
<code>production_mrr_coverage</code>	<code>float</code>	MRR de production moyenné sur <i>toutes</i> les exécutions (0 pour celles sans hit).
<code>exec2code</code>	<code>dict[str, int]</code>	<code>executionId</code> (<code>string</code>) → code contigu, sur toutes les exécutions du split.
<code>code2exec</code>	<code>list</code>	code contigu → <code>executionId</code> (inverse de <code>exec2code</code>).

Le commentaire final du code précise que `df_positives_rank`, `negative_candidates` et `df_all_candidates` ne sont plus exposés depuis cette fonction : les tables prototype de production sont reconstruites indépendamment en aval.

6 Assemblage final du graphe PyG

6.1 build_pyg_data

Regroupe le graphe de message-passing et les tenseurs de supervision des trois splits dans un unique objet `torch_geometric.data.Data`.

Signature.

```
def build_pyg_data(
    num_nodes: int,
    train_edge_index: torch.Tensor,
    train_neg_edge_index: torch.Tensor,
    train_pos_edge_index: torch.Tensor,
    train_edge_attr: torch.Tensor,
    val_pos_edge_index: torch.Tensor,
    val_neg_edge_index: torch.Tensor,
    test_pos_edge_index: torch.Tensor,
    test_neg_edge_index: torch.Tensor,
    x: torch.Tensor = None,
) -> Data:
```

Arguments.

Argument	Type	Rôle
<code>num_nodes</code>	<code>int</code>	Nombre total de nœuds, cf <code>count_nodes</code> (2.7).
<code>train_edge_index</code>	<code>torch.Tensor</code>	Graphe de message-passing (arêtes de co-vue de <code>train</code> uniquement), cf 3.3.
<code>train_edge_attr</code>	<code>torch.Tensor</code>	Poids associés à <code>train_edge_index</code> , cf 3.3.
<code>train_pos_edge_index</code> , <code>train_neg_edge_index</code>	<code>torch.Tensor</code>	Arêtes de supervision positives / négatives du split <code>train</code> , cf <code>pos_edge_index</code> / <code>neg_edge_index</code> de 5.1.
<code>val_pos_edge_index</code> , <code>val_neg_edge_index</code>	<code>torch.Tensor</code>	Idem pour le split <code>val</code> .
<code>test_pos_edge_index</code> , <code>test_neg_edge_index</code>	<code>torch.Tensor</code>	Idem pour le split <code>test</code> .
<code>x</code>	<code>torch.Tensor</code> , défaut <code>None</code>	Matrice d'attributs de nœuds, optionnelle.

Traitement.

- `data: Data(edge_index=train_edge_index, edge_attr=train_edge_attr, num_nodes=num_nodes)`
 - graphe de message-passing.
- Si `x` n'est pas `None`, assigné à `data.x`.
- Les six tenseurs de supervision (`train/val/test × pos/neg`) sont attachés à `data` comme attributs additionnels, sans transformation.

Sortie. Retourne un objet `Data` unique, pas un tenseur ni un `DataFrame` :

Attribut		Contenu
<code>edge_index</code> / <code>edge_attr</code> / <code>num_nodes</code> / <code>x</code>		Graphe de message-passing (arêtes et poids de train , nombre de nœuds). Attributs de nœuds, présent seulement si fourni en entrée.
<code>train_pos_edge_index</code> / <code>train_neg_edge_index</code>		Supervision positive / négative du split train .
<code>val_pos_edge_index</code> / <code>val_neg_edge_index</code>		Idem pour val .
<code>test_pos_edge_index</code> / <code>test_neg_edge_index</code>		Idem pour test .

Ce Data suit la convention transductive usuelle des GNN de prédiction de liens : le graphe de message-passing ne porte que les arêtes de **train** — **val/test** n'apportent que des arêtes de supervision, jamais de structure supplémentaire au graphe.